

```
// File: backend/app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.api.auth import router as auth_router
from app.api.builds import router as build_router
from app.api.catalog import router as catalog_router
from app.api.community import router as community_router
from app.api.compat import router as compat_router
from app.api.health import create_health_router
from app.api.ops import router as ops_router
from app.api.perf import router as perf_router
from app.api.profile import router as profile_router
from app.api.progress import create_progress_router
from app.api.review import router as review_router
from app.api.saved_builds import router as saved_builds_router
from app.api.upgrade import router as upgrade_router
from app.core.config import Settings
from app.core.rate_limit import create_rate_limiter
from app.core.request_size import RequestBodySizeLimitMiddleware
from app.core.response_cache import ResponseCache
from app.core.security_headers import SecurityHeadersMiddleware
from app.core.usage_metrics import HighCostUsageMetrics
def create_app(settings: Settings) -> FastAPI:
    if settings.app_env == "production" and settings.auth_token_secret == "dev-only-change-me":
        raise RuntimeError("APP_ENV=production 下禁止使用默认 auth_token_secret，请配置生产密钥后再启动。")
    app = FastAPI(
        title="AI PC Builder API",
        version="0.1.0",
        docs_url="/docs" if settings.api_docs_enabled else None,
        redoc_url="/redoc" if settings.api_docs_enabled else None,
        openapi_url="/openapi.json" if settings.api_docs_enabled else None,
    )
    if settings.cors_origin_list:
        app.add_middleware(
            CORSMiddleware,
            allow_origins=settings.cors_origin_list,
            allow_credentials=True,
            allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
            allow_headers=["Authorization", "Content-Type"],
        )
    app.add_middleware(
        RequestBodySizeLimitMiddleware,
        max_body_bytes=settings.max_request_body_bytes,
    )
    app.add_middleware(SecurityHeadersMiddleware)
    app.state.settings = settings
    app.state.high_cost_rate_limiter = create_rate_limiter(
        max_requests=settings.high_cost_rate_limit_max_requests,
        window_seconds=settings.high_cost_rate_limit_window_seconds,
        namespace="high-cost",
    )
```

```
        redis_url=settings.redis_url,
    )
    app.state.auth_sms_rate_limiter = create_rate_limiter(
        max_requests=settings.auth_sms_rate_limit_max_requests,
        window_seconds=settings.auth_rate_limit_window_seconds,
        namespace="auth-sms",
        redis_url=settings.redis_url,
    )
    app.state.auth_login_rate_limiter = create_rate_limiter(
        max_requests=settings.auth_login_rate_limit_max_requests,
        window_seconds=settings.auth_rate_limit_window_seconds,
        namespace="auth-login",
        redis_url=settings.redis_url,
    )
    app.state.community_write_rate_limiter = create_rate_limiter(
        max_requests=settings.community_write_rate_limit_max_requests,
        window_seconds=settings.community_write_rate_limit_window_seconds,
        namespace="community-write",
        redis_url=settings.redis_url,
    )
    app.state.high_cost_usage_metrics = HighCostUsageMetrics()
    app.state.response_cache = ResponseCache(
        ttl_seconds=settings.response_cache_ttl_seconds,
        max_entries=settings.response_cache_max_entries,
    )
    app.include_router(auth_router)
    app.include_router(build_router)
    app.include_router(catalog_router)
    app.include_router(community_router)
    app.include_router(compat_router)
    app.include_router(ops_router)
    app.include_router(profile_router)
    app.include_router(review_router)
    app.include_router(saved_builds_router)
    app.include_router(upgrade_router)
    app.include_router(perf_router)
    app.include_router(create_health_router(settings))
    app.include_router(create_progress_router())
    return app

app = create_app(Settings())
// File: backend/app/core/config.py
import ipaddress
from typing import Optional
from urllib.parse import urlparse
from pydantic import AliasChoices, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_prefix="APP_",
```

```

        extra="ignore",
    )
    service_name: str = "ai-pc-builder-api"
    app_env: str = Field(default="development", validation_alias=AliasChoices("APP_ENV", "app_env"))
    postgres_url: Optional[str] = None
    redis_url: Optional[str] = None
    auth_token_secret: str = "dev-only-change-me"
    auth_sms_debug: bool = True
    ai_provider_api_key: Optional[str] = None
    ai_provider_base_url: str = "https://api.deepseek.com"
    ai_model: str = "deepseek-v4-flash"
    ai_provider_timeout_seconds: float = 8.0
    high_cost_rate_limit_enabled: bool = True
    high_cost_rate_limit_max_requests: int = 30
    high_cost_rate_limit_window_seconds: int = 60
    high_cost_estimated_cost_cents: int = 0
    auth_rate_limit_enabled: bool = True
    auth_sms_rate_limit_max_requests: int = 5
    auth_login_rate_limit_max_requests: int = 10
    auth_rate_limit_window_seconds: int = 60
    response_cache_enabled: bool = True
    response_cache_ttl_seconds: int = 300
    response_cache_max_entries: int = 512
    max_request_body_bytes: int = 10_000_000
    ops_token: Optional[str] = None
    community_image_upload_enabled: bool = False
    community_image_max_bytes: int = 5_000_000
    community_image_oss_bucket: Optional[str] = None
    community_image_oss_region: Optional[str] = None
    community_image_oss_access_key_id: Optional[str] = None
    community_image_oss_access_key_secret: Optional[str] = None
    community_image_oss_security_token: Optional[str] = None
    community_write_rate_limit_enabled: bool = True
    community_write_rate_limit_max_requests: int = 60
    community_write_rate_limit_window_seconds: int = 60
    apple_login_client_id: Optional[str] = None
    cors_allowed_origins: str = ""
    api_docs_enabled: bool = False
    production_data_readiness_required: bool = False
    @property
    def sqlalchemy_database_url(self) -> Optional[str]:
        if self.postgres_url and self.postgres_url.startswith("postgresql://"):
            return self.postgres_url.replace("postgresql://", "postgresql+psycopg://", 1)
        return self.postgres_url
    @property
    def ai_provider_chat_url(self) -> str:
        return self.ai_provider_base_url.rstrip("/") + "/chat/completions"
    @property
    def ai_provider_base_url_status(self) -> str:
        return _external_https_url_status(self.ai_provider_base_url)

```

```

@property
def cors_origin_list(self) -> list[str]:
    origins = self.raw_cors_origin_list
    if any(_cors_origin_status(origin) != "configured" for origin in origins):
        return []
    return origins

@property
def raw_cors_origin_list(self) -> list[str]:
    return [origin.strip() for origin in self.cors_allowed_origins.split(",") if origin.strip()]

@property
def cors_allowed_origins_status(self) -> str:
    origins = self.raw_cors_origin_list
    if not origins:
        return "not_configured"
    if any(_cors_origin_status(origin) != "configured" for origin in origins):
        return "invalid"
    return "configured"

@property
def community_image_upload_configured(self) -> bool:
    return all(
        [
            self.community_image_upload_enabled,
            self.community_image_oss_bucket,
            self.community_image_oss_region,
            self.community_image_oss_access_key_id,
            self.community_image_oss_access_key_secret,
        ]
    )

@property
def community_image_oss_host(self) -> Optional[str]:
    if not self.community_image_oss_bucket or not self.community_image_oss_region:
        return None
    return (
        f"https://{self.community_image_oss_bucket}"
        f".oss-{self.community_image_oss_region}.aliyuncs.com"
    )

def _cors_origin_status(origin: str) -> str:
    if origin == "":
        return "invalid"
    parsed = urlparse(origin)
    if parsed.scheme not in {"http", "https"}:
        return "invalid"
    if not parsed.netloc:
        return "invalid"
    if parsed.path not in {"", "/" } or parsed.params or parsed.query or parsed.fragment:
        return "invalid"
    return "configured"

def _external_https_url_status(value: str) -> str:
    parsed = urlparse(value)
    if parsed.scheme != "https":

```

```

    return "invalid"
if not parsed.hostname:
    return "invalid"
hostname = parsed.hostname.lower()
if hostname in {"localhost", "localhost.localdomain"}:
    return "invalid"
try:
    address = ipaddress.ip_address(hostname)
except ValueError:
    return "configured"
if (
    address.is_private
    or address.is_loopback
    or address.is_link_local
    or address.is_reserved
    or address.is_multicast
    or address.is_unspecified
):
    return "invalid"
return "configured"
// File: backend/app/core/rate_limit.py
import hashlib
import time
from dataclasses import dataclass
from typing import Any, Dict, Optional, Union
from fastapi import Header, HTTPException, Request
from app.auth.models import Account
@dataclass
class RateLimitBucket:
    window_started_at: float
    count: int
class FixedWindowRateLimiter:
    def __init__(self, max_requests: int, window_seconds: int) -> None:
        self.max_requests = max_requests
        self.window_seconds = window_seconds
        self._buckets: Dict[str, RateLimitBucket] = {}
    def check(self, key: str, now: Optional[float] = None) -> Optional[int]:
        if self.max_requests <= 0 or self.window_seconds <= 0:
            return None
        current_time = now if now is not None else time.monotonic()
        bucket = self._buckets.get(key)
        if bucket is None or current_time - bucket.window_started_at >= self.window_seconds:
            self._buckets[key] = RateLimitBucket(
                window_started_at=current_time,
                count=1,
            )
            return None
        if bucket.count >= self.max_requests:
            retry_after = self.window_seconds - (current_time - bucket.window_started_at)
            return max(1, int(retry_after))

```

```

        bucket.count += 1

        return None

class RedisFixedWindowRateLimiter:
    def __init__(
        self,
        max_requests: int,
        window_seconds: int,
        namespace: str,
        redis_client: Any,
    ) -> None:
        self.max_requests = max_requests
        self.window_seconds = window_seconds
        self.namespace = namespace
        self.redis_client = redis_client
        self._fallback_limiter = FixedWindowRateLimiter(
            max_requests=max_requests,
            window_seconds=window_seconds,
        )

    def check(self, key: str, now: Optional[float] = None) -> Optional[int]:
        if self.max_requests <= 0 or self.window_seconds <= 0:
            return None

        current_time = now if now is not None else time.time()
        bucket = int(current_time // self.window_seconds)
        redis_key = f"rate-limit:{self.namespace}:{key}:{bucket}"

        try:
            count = int(self.redis_client.incr(redis_key))
            if count == 1:
                self.redis_client.expire(redis_key, self.window_seconds)
            if count <= self.max_requests:
                return None

            ttl = int(self.redis_client.ttl(redis_key))
            if ttl <= 0:
                ttl = self.window_seconds
            return max(1, ttl)
        except Exception:
            return self._fallback_limiter.check(key, now=current_time)

def create_rate_limiter(
    max_requests: int,
    window_seconds: int,
    namespace: str,
    redis_url: Optional[str],
) -> Union[FixedWindowRateLimiter, RedisFixedWindowRateLimiter]:
    if redis_url:
        try:
            from redis import Redis
        except ImportError:
            return FixedWindowRateLimiter(max_requests=max_requests, window_seconds=window_seconds)
        try:
            redis_client = Redis.from_url(redis_url, decode_responses=True)
        except Exception:

```

```

        return FixedWindowRateLimiter(max_requests=max_requests, window_seconds=window_seconds)
    return RedisFixedWindowRateLimiter(
        max_requests=max_requests,
        window_seconds=window_seconds,
        namespace=namespace,
        redis_client=redis_client,
    )
    return FixedWindowRateLimiter(max_requests=max_requests, window_seconds=window_seconds)
def high_cost_rate_limit(
    request: Request,
    authorization: Optional[str] = Header(default=None),
) -> None:
    settings = request.app.state.settings
    endpoint = request.url.path
    if not settings.high_cost_rate_limit_enabled:
        request.app.state.high_cost_usage_metrics.record_request(endpoint)
        return
    limiter: FixedWindowRateLimiter = request.app.state.high_cost_rate_limiter
    retry_after = limiter.check(_high_cost_key(request, authorization))
    if retry_after is not None:
        request.app.state.high_cost_usage_metrics.record_rate_limited(endpoint)
        raise HTTPException(
            status_code=429,
            detail="Rate limit exceeded",
            headers={"Retry-After": str(retry_after)},
        )
    request.app.state.high_cost_usage_metrics.record_request(endpoint)
def auth_sms_rate_limit(request: Request) -> None:
    _check_named_limiter(
        request=request,
        limiter_name="auth_sms_rate_limiter",
        key_prefix="sms",
        detail="SMS rate limit exceeded",
    )
def auth_login_rate_limit(request: Request) -> None:
    _check_named_limiter(
        request=request,
        limiter_name="auth_login_rate_limiter",
        key_prefix="login",
        detail="Login rate limit exceeded",
    )
def community_write_rate_limit(request: Request, account: Account) -> None:
    settings = request.app.state.settings
    if not settings.community_write_rate_limit_enabled:
        return
    limiter: FixedWindowRateLimiter = request.app.state.community_write_rate_limiter
    retry_after = limiter.check(f"community-write:{account.id}")
    if retry_after is not None:
        raise HTTPException(
            status_code=429,

```

```

        detail="Community write rate limit exceeded",
        headers={"Retry-After": str(retry_after)},
    )
def _check_named_limiter(
    request: Request,
    limiter_name: str,
    key_prefix: str,
    detail: str,
) -> None:
    settings = request.app.state.settings
    if not settings.auth_rate_limit_enabled:
        return
    limiter: FixedWindowRateLimiter = getattr(request.app.state, limiter_name)
    retry_after = limiter.check(f"{key_prefix}:{_client_host(request)}")
    if retry_after is not None:
        raise HTTPException(
            status_code=429,
            detail=detail,
            headers={"Retry-After": str(retry_after)},
        )
def _high_cost_key(request: Request, authorization: Optional[str]) -> str:
    if authorization and authorization.startswith("Bearer "):
        token = authorization.removeprefix("Bearer ").strip()
        digest = hashlib.sha256(token.encode("utf-8")).hexdigest()
        return f"token:{digest}"
    return f"ip:{_client_host(request)}"
def _client_host(request: Request) -> str:
    return request.client.host if request.client else "unknown"
// File: backend/app/core/response_cache.py
import hashlib
import json
import time
from dataclasses import dataclass
from typing import Any, Dict, Optional
from pydantic import BaseModel
@dataclass
class ResponseCacheEntry:
    stored_at: float
    value: Dict[str, Any]
class ResponseCache:
    def __init__(self, ttl_seconds: int, max_entries: int) -> None:
        self.ttl_seconds = ttl_seconds
        self.max_entries = max_entries
        self._entries: Dict[str, ResponseCacheEntry] = {}
    def get(self, key: str, now: Optional[float] = None) -> Optional[Dict[str, Any]]:
        if self.ttl_seconds <= 0 or self.max_entries <= 0:
            return None
        current_time = now if now is not None else time.monotonic()
        entry = self._entries.get(key)
        if entry is None:

```



```

        return None

    if current_time - entry.stored_at >= self.ttl_seconds:
        self._entries.pop(key, None)
        return None

    return dict(entry.value)

def set(self, key: str, value: Dict[str, Any], now: Optional[float] = None) -> None:
    if self.ttl_seconds <= 0 or self.max_entries <= 0:
        return

    if len(self._entries) >= self.max_entries and key not in self._entries:
        oldest_key = min(
            self._entries,
            key=lambda entry_key: self._entries[entry_key].stored_at,
        )
        self._entries.pop(oldest_key, None)

    current_time = now if now is not None else time.monotonic()
    self._entries[key] = ResponseCacheEntry(stored_at=current_time, value=dict(value))

def response_cache_key(namespace: str, payload: BaseModel) -> str:
    raw = json.dumps(
        payload.model_dump(mode="json"),
        ensure_ascii=False,
        separators=(",", ":"),
        sort_keys=True,
    )
    digest = hashlib.sha256(raw.encode("utf-8")).hexdigest()
    return f"{namespace}:{digest}"

// File: backend/app/compat/engine.py
from typing import Dict, List, Literal, Optional
from pydantic import BaseModel, Field
from app.catalog.models import HardwareComponent
FindingLevel = Literal["pass", "warning", "error"]
COMPATIBILITY_RULE_VERSION = "2026-06-16"
REQUIRED_ROLES = {
    "cpu": "CPU",
    "motherboard": "主板",
    "ram": "内存",
    "psu": "电源",
}

class CompatibilityRule(BaseModel):
    code: str
    title: str
    description: str
    level_when_failed: FindingLevel

COMPATIBILITY_RULES = [
    CompatibilityRule(
        code="unknown_component",
        title="硬件库型号校验",
        description="所有传入型号必须能在维护中的硬件库里找到。",
        level_when_failed="error",
    ),
    CompatibilityRule(

```

UzBox ai 装机助手 V1.0

```

        code="missing_required",
        title="核心配件完整性",
        description="CPU、主板、内存和电源是完整兼容性判断的必需项。",
        level_when_failed="error",
    ),
    CompatibilityRule(
        code="cpu_motherboard_socket",
        title="CPU 和主板插槽",
        description="CPU socket 必须和主板 socket 一致。",
        level_when_failed="error",
    ),
    CompatibilityRule(
        code="motherboard_ram_type",
        title="主板和内存代际",
        description="主板内存代际必须和内存类型一致。",
        level_when_failed="error",
    ),
    CompatibilityRule(
        code="psu_headroom",
        title="电源余量",
        description="电源瓦数需要覆盖 CPU/GPU 等配件的估算功耗并留有余量。",
        level_when_failed="warning",
    ),
    CompatibilityRule(
        code="cpu_gpu_balance",
        title="CPU/GPU 搭配均衡",
        description="用 perf_index 检查高 U 低显或低 U 高显。",
        level_when_failed="warning",
    ),
    CompatibilityRule(
        code="cpu_motherboard_tier",
        title="CPU 和主板档次",
        description="高端 CPU 搭配入门主板时给出稳定性和扩展性提醒。",
        level_when_failed="warning",
    ),
]

class BuildSelection(BaseModel):
    components: Dict[str, str] = Field(max_length=24)
    def component_id(self, role: str) -> Optional[str]:
        if role == "psu":
            return self.components.get("psu") or self.components.get("power_supply")
        return self.components.get(role)
    def selected_ids(self) -> List[str]:
        return [component_id for component_id in self.components.values() if component_id]

class CompatibilityFinding(BaseModel):
    level: FindingLevel
    code: str
    title: str
    detail: str
    component_ids: List[str]

```

```

class CompatibilityResult(BaseModel):
    compatible: bool
    summary: str
    findings: List[CompatibilityFinding]
    finding_counts: Dict[FindingLevel, int]
    checked_rule_codes: List[str]

class CompatibilityRulesResponse(BaseModel):
    version: str
    rules: List[CompatibilityRule]

def evaluate_compatibility(
    selection: BuildSelection,
    components_by_id: Dict[str, HardwareComponent],
) -> CompatibilityResult:
    findings: List[CompatibilityFinding] = []
    findings.extend(_unknown_component_findings(selection, components_by_id))
    findings.extend(_missing_required_findings(selection))
    findings.extend(_socket_findings(selection, components_by_id))
    findings.extend(_ram_type_findings(selection, components_by_id))
    findings.extend(_psu_headroom_findings(selection, components_by_id))
    findings.extend(_balance_findings(selection, components_by_id))
    findings.extend(_motherboard_tier_findings(selection, components_by_id))
    compatible = not any(finding.level == "error" for finding in findings)
    summary = (
        "这套配置没有发现硬性兼容问题。"
        if compatible
        else "这套配置有硬性兼容问题，需要先调整。"
    )
    return CompatibilityResult(
        compatible=compatible,
        summary=summary,
        findings=findings,
        finding_counts=_finding_counts(findings),
        checked_rule_codes=_checked_rule_codes(findings),
    )

def compatibility_rules_response() -> CompatibilityRulesResponse:
    return CompatibilityRulesResponse(version=COMPATIBILITY_RULE_VERSION, rules=COMPATIBILITY_RULES)

def _unknown_component_findings(
    selection: BuildSelection,
    components_by_id: Dict[str, HardwareComponent],
) -> List[CompatibilityFinding]:
    findings: List[CompatibilityFinding] = []
    for component_id in selection.selected_ids():
        if component_id not in components_by_id:
            findings.append(
                CompatibilityFinding(
                    level="error",
                    code="unknown_component",
                    title="硬件库找不到这个型号",
                    detail=f"没有在硬件库里找到 {component_id}，不能用它做兼容性判断。",
                    component_ids=[component_id],
                )
            )

```

```

    )
    )
    return findings

def _missing_required_findings(selection: BuildSelection) -> List[CompatibilityFinding]:
    findings: List[CompatibilityFinding] = []
    for role, label in REQUIRED_ROLES.items():
        if not selection.component_id(role):
            findings.append(
                CompatibilityFinding(
                    level="error",
                    code=f"missing_{role}",
                    title=f"缺少{label}",
                    detail=f"这套配置还没有选择{label}，不能完整判断兼容性。",
                    component_ids=[],
                )
            )
    return findings

def _socket_findings(
    selection: BuildSelection,
    components_by_id: Dict[str, HardwareComponent],
) -> List[CompatibilityFinding]:
    cpu = _component_for_role(selection, components_by_id, "cpu")
    motherboard = _component_for_role(selection, components_by_id, "motherboard")
    if cpu is None or motherboard is None:
        return []
    cpu_socket = _string_spec(cpu, "socket")
    motherboard_socket = _string_spec(motherboard, "socket")
    if not cpu_socket or not motherboard_socket:
        return []
    if cpu_socket != motherboard_socket:
        return [
            CompatibilityFinding(
                level="error",
                code="cpu_motherboard_socket",
                title="CPU 和主板插槽不匹配",
                detail=f"{cpu.name} 是 {cpu_socket}，{motherboard.name} 是 {motherboard_socket}，插槽不匹配。",
                component_ids=[cpu.id, motherboard.id],
            )
        ]
    return [
        CompatibilityFinding(
            level="pass",
            code="cpu_motherboard_socket",
            title="CPU 和主板插槽匹配",
            detail=f"{cpu.name} 可以安装在 {motherboard.name} 上。",
            component_ids=[cpu.id, motherboard.id],
        )
    ]

def _ram_type_findings(
    selection: BuildSelection,

```

```

components_by_id: Dict[str, HardwareComponent],
) -> List[CompatibilityFinding]:
    motherboard = _component_for_role(selection, components_by_id, "motherboard")
    ram = _component_for_role(selection, components_by_id, "ram")
    if motherboard is None or ram is None:
        return []
    motherboard_mem_type = _string_spec(motherboard, "mem_type")
    ram_type = _string_spec(ram, "type")
    if not motherboard_mem_type or not ram_type:
        return []
    if motherboard_mem_type != ram_type:
        return [
            CompatibilityFinding(
                level="error",
                code="motherboard_ram_type",
                title="主板和内存代际不匹配",
                detail=f"{motherboard.name} 需要 {motherboard_mem_type} 内存，但你选的是 {ram_type}。",
                component_ids=[motherboard.id, ram.id],
            )
        ]
    return [
        CompatibilityFinding(
            level="pass",
            code="motherboard_ram_type",
            title="主板和内存代际匹配",
            detail=f"{motherboard.name} 和 {ram.name} 的内存代际匹配。",
            component_ids=[motherboard.id, ram.id],
        )
    ]

def _psu_headroom_findings(
    selection: BuildSelection,
    components_by_id: Dict[str, HardwareComponent],
) -> List[CompatibilityFinding]:
    psu = _component_for_role(selection, components_by_id, "psu")
    if psu is None:
        return []
    psu_watt = _int_spec(psu, "watt")
    total_tdp = sum(
        _int_spec(components_by_id[component_id], "tdp") or 0
        for component_id in set(selection.selected_ids())
        if component_id in components_by_id
        and components_by_id[component_id].category != "psu"
    )
    if not psu_watt or total_tdp <= 0:
        return []
    if psu_watt < total_tdp:
        return [
            CompatibilityFinding(
                level="error",
                code="psu_headroom",

```

```

        title="电源瓦数不够",
        detail=f"预计功耗约 {total_tdp}W, {psu_watt}W 电源可能带不动这套配置。",
        component_ids=[psu.id],
    )
]
if psu_watt < total_tdp * 1.3:
    return [
        CompatibilityFinding(
            level="warning",
            code="psu_headroom",
            title="电源余量偏紧",
            detail=f"预计功耗约 {total_tdp}W, {psu_watt}W 电源余量偏紧, 建议换更高瓦数。",
            component_ids=[psu.id],
        )
    ]
return [
    CompatibilityFinding(
        level="pass",
        code="psu_headroom",
        title="电源余量正常",
        detail=f"预计功耗约 {total_tdp}W, {psu_watt}W 电源有基础余量。",
        component_ids=[psu.id],
    )
]
def _balance_findings(
    selection: BuildSelection,
    components_by_id: Dict[str, HardwareComponent],
) -> List[CompatibilityFinding]:
    cpu = _component_for_role(selection, components_by_id, "cpu")
    gpu = _component_for_role(selection, components_by_id, "gpu")
    if cpu is None or gpu is None:
        return []
    cpu_perf = _int_spec(cpu, "perf_index")
    gpu_perf = _int_spec(gpu, "perf_index")
    if cpu_perf <= 0 or gpu_perf <= 0:
        return []
    if cpu_perf >= gpu_perf * 1.6:
        return [
            CompatibilityFinding(
                level="warning",
                code="cpu_gpu_balance",
                title="CPU 明显强于显卡",
                detail=f"CPU 明显强于显卡: {cpu.name} 的性能档位高于 {gpu.name}, 游戏场景可能是高 U 低显。",
                component_ids=[cpu.id, gpu.id],
            )
        ]
    if gpu_perf >= cpu_perf * 1.8:
        return [
            CompatibilityFinding(
                level="warning",

```

```

        code="cpu_gpu_balance",
        title="显卡明显强于 CPU",
        detail=f"{gpu.name} 的性能档位明显高于 {cpu.name}，高帧率游戏可能被 CPU 拖住。",
        component_ids=[cpu.id, gpu.id],
    )
]
return [
    CompatibilityFinding(
        level="pass",
        code="cpu_gpu_balance",
        title="CPU 和显卡搭配均衡",
        detail=f"{cpu.name} 和 {gpu.name} 的性能档位没有明显失衡。",
        component_ids=[cpu.id, gpu.id],
    )
]
def _motherboard_tier_findings(
    selection: BuildSelection,
    components_by_id: Dict[str, HardwareComponent],
) -> List[CompatibilityFinding]:
    cpu = _component_for_role(selection, components_by_id, "cpu")
    motherboard = _component_for_role(selection, components_by_id, "motherboard")
    if cpu is None or motherboard is None:
        return []
    cpu_perf = _int_spec(cpu, "perf_index")
    chipset = _string_spec(motherboard, "chipset").upper()
    if cpu_perf <= 0 or not chipset:
        return []
    if cpu_perf >= 90 and chipset.startswith(("H", "A")):
        return [
            CompatibilityFinding(
                level="warning",
                code="cpu_motherboard_tier",
                title="高端 CPU 搭配入门主板",
                detail=f"{motherboard.name} 属于入门主板，搭配 {cpu.name} 这类高性能 CPU 时供电和扩展性偏保守。",
                component_ids=[cpu.id, motherboard.id],
            )
        ]
    return []
def _finding_counts(findings: List[CompatibilityFinding]) -> Dict[FindingLevel, int]:
    return {
        "pass": sum(finding.level == "pass" for finding in findings),
        "warning": sum(finding.level == "warning" for finding in findings),
        "error": sum(finding.level == "error" for finding in findings),
    }
def _checked_rule_codes(findings: List[CompatibilityFinding]) -> List[str]:
    return sorted({finding.code for finding in findings})
def _component_for_role(
    selection: BuildSelection,
    components_by_id: Dict[str, HardwareComponent],
    role: str,

```

```

) -> Optional[HardwareComponent]:
    component_id = selection.component_id(role)
    if not component_id:
        return None
    return components_by_id.get(component_id)
def _string_spec(component: HardwareComponent, key: str) -> str:
    value = component.specs.get(key)
    return value if isinstance(value, str) else ""
def _int_spec(component: HardwareComponent, key: str) -> int:
    value = component.specs.get(key)
    return value if isinstance(value, int) else 0
// File: backend/app/api/compat.py
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from app.catalog.repository import get_components_by_ids
from app.compat.engine import (
    BuildSelection,
    CompatibilityResult,
    CompatibilityRulesResponse,
    compatibility_rules_response,
    evaluate_compatibility,
)
from app.db import get_session
router = APIRouter(prefix="/v1/compat", tags=["compatibility"])
@router.get("/rules", response_model=CompatibilityRulesResponse)
def compatibility_rules() -> CompatibilityRulesResponse:
    return compatibility_rules_response()
@router.post("/check", response_model=CompatibilityResult)
def check_compatibility(
    selection: BuildSelection,
    session: Session = Depends(get_session),
) -> CompatibilityResult:
    components = get_components_by_ids(session, selection.selected_ids())
    return evaluate_compatibility(selection, {component.id: component for component in components})
// File: backend/app/api/catalog.py
from datetime import datetime
from typing import Any, Dict, List, Optional
from fastapi import APIRouter, Depends, HTTPException, Query
from pydantic import BaseModel
from sqlalchemy.orm import Session
from app.catalog.models import ComponentPrice, HardwareComponent
from app.catalog.readiness import DataReadiness, build_data_readiness
from app.catalog.repository import (
    get_component_price,
    list_component_prices,
    list_components,
    list_compatible_motherboards,
)
from app.db import get_session
class HardwareComponentResponse(BaseModel):

```



```

id: str
category: str
name: str
brand: str
detail_raw: str
specs: Dict[str, Any]
is_recommended: bool
status: str
class ComponentPriceResponse(BaseModel):
    component_id: str
    reference_price: int
    price_range_low: Optional[int]
    price_range_high: Optional[int]
    source: str
    accepted_count: int
    rejected_count: int
    review_reasons: List[str]
    approved_at: datetime
class ComponentPriceDetailResponse(ComponentPriceResponse):
    pass
class CatalogReadinessResponse(BaseModel):
    ready: bool
    component_count: int
    price_count: int
    active_template_count: int
    recommended_counts: Dict[str, int]
    priced_recommended_counts: Dict[str, int]
    missing_recommended_categories: List[str]
    missing_priced_recommended_categories: List[str]
router = APIRouter(prefix="/v1/catalog", tags=["catalog"])
@router.get("/components", response_model=List[HardwareComponentResponse])
def components(
    category: Optional[str] = None,
    brand: Optional[str] = None,
    q: Optional[str] = None,
    limit: int = Query(default=100, ge=1, le=500),
    offset: int = Query(default=0, ge=0),
    session: Session = Depends(get_session),
) -> List[HardwareComponent]:
    return list_components(session, category=category, brand=brand, q=q, limit=limit, offset=offset)
@router.get("/motherboards", response_model=List[HardwareComponentResponse])
def motherboards(
    cpu: str,
    session: Session = Depends(get_session),
) -> List[HardwareComponent]:
    return list_compatible_motherboards(session, cpu)
@router.get("/prices", response_model=List[ComponentPriceResponse])
def prices(
    limit: int = Query(default=100, ge=1, le=500),
    offset: int = Query(default=0, ge=0),

```

UzBox ai 装机助手 V1.0

```

    session: Session = Depends(get_session),
) -> List[ComponentPrice]:
    return list_component_prices(session, limit=limit, offset=offset)
@router.get("/readiness", response_model=CatalogReadinessResponse)
def readiness(
    session: Session = Depends(get_session),
) -> DataReadiness:
    return build_data_readiness(session)
@router.get("/components/{component_id}/price", response_model=ComponentPriceDetailResponse)
def component_price(
    component_id: str,
    session: Session = Depends(get_session),
) -> ComponentPrice:
    price = get_component_price(session, component_id)
    if price is None:
        raise HTTPException(status_code=404, detail="Component price not found")
    return price
// File: backend/app/perf/profiles.py
from dataclasses import dataclass
from enum import Enum
from typing import Dict, Optional
class GameLoadType(str, Enum):
    CPU = "cpu"
    GPU = "gpu"
    MIXED = "mixed"
class RenderMode(str, Enum):
    NATIVE = "native"
    DLSS_QUALITY = "dlss_quality"
    DLSS_QUALITY_FG = "dlss_quality_fg"
    FSR_QUALITY = "fsr_quality"
    FSR_QUALITY_FG = "fsr_quality_fg"
@dataclass(frozen=True)
class GameProfile:
    game_id: str
    load_type: GameLoadType
    supports_dlss: bool = False
    supports_fsr: bool = False
    supports_frame_generation: bool = False
    fps_cap: Optional[int] = None
@dataclass(frozen=True)
class GPUCapabilities:
    supports_dlss: bool
    supports_fsr: bool
    supports_standard_frame_generation: bool
APPROVED_GAME_PROFILES: Dict[str, GameProfile] = {
    "valorant": GameProfile("valorant", GameLoadType.CPU),
    "cs2": GameProfile("cs2", GameLoadType.CPU, supports_fsr=True),
    "pubg": GameProfile("pubg", GameLoadType.MIXED, supports_dlss=True),
    "delta-force": GameProfile(
        "delta-force",

```

```

        GameLoadType.MIXED,
        supports_dlss=True,
        supports_fsr=True,
        supports_frame_generation=True,
    ),
    "teamfight-tactics": GameProfile("teamfight-tactics", GameLoadType.CPU),
    "league-of-legends": GameProfile("league-of-legends", GameLoadType.CPU),
    "call-of-duty-warzone": GameProfile(
        "call-of-duty-warzone",
        GameLoadType.MIXED,
        supports_dlss=True,
        supports_fsr=True,
        supports_frame_generation=True,
    ),
    "cyberpunk-2077": GameProfile(
        "cyberpunk-2077",
        GameLoadType.GPU,
        supports_dlss=True,
        supports_fsr=True,
        supports_frame_generation=True,
    ),
    "red-dead-redemption-2": GameProfile(
        "red-dead-redemption-2",
        GameLoadType.GPU,
        supports_dlss=True,
        supports_fsr=True,
    ),
    "gta-v": GameProfile("gta-v", GameLoadType.MIXED),
    "black-myth-wukong": GameProfile(
        "black-myth-wukong",
        GameLoadType.GPU,
        supports_dlss=True,
        supports_fsr=True,
        supports_frame_generation=True,
    ),
    "forza-horizon-6": GameProfile("forza-horizon-6", GameLoadType.GPU),
    "elden-ring": GameProfile("elden-ring", GameLoadType.GPU, fps_cap=60),
    "cities-skylines": GameProfile("cities-skylines", GameLoadType.CPU),
    "minecraft-java-edition": GameProfile(
        "minecraft-java-edition",
        GameLoadType.CPU,
    ),
}

def render_mode_for(profile: GameProfile, gpu: GPUCapabilities) -> RenderMode:
    if profile.supports_dlss and gpu.supports_dlss:
        if (
            profile.supports_frame_generation
            and gpu.supports_standard_frame_generation
        ):
            return RenderMode.DLSS_QUALITY_FG

```

```
        return RenderMode.DLSS_QUALITY
    if profile.supports_fsr and gpu.supports_fsr:
        if (
            profile.supports_frame_generation
            and gpu.supports_standard_frame_generation
        ):
            return RenderMode.FSR_QUALITY_FG
        return RenderMode.FSR_QUALITY
    return RenderMode.NATIVE
// File: backend/app/api/builds.py
import logging
from typing import Optional
from uuid import UUID
from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import ValidationError
from sqlalchemy.orm import Session
from app.builds.ai_provider import AIProviderError, select_build_with_deepseek
from app.builds.gpu_rules import gpu_brand_allowed_for_budget
from app.builds.customization import (
    CustomizationError,
    customization_candidates,
    customize_template,
    deterministic_customization,
    find_owned_gpu,
)
from app.builds.repository import list_build_templates
from app.builds.selection_cache import (
    current_cache_version,
    mark_option_selected,
    option_cache_key,
    request_identity,
    selected_option,
    store_pending_option,
)
from app.builds.service import (
    BuildGenerationResponse,
    BuildOptionResponse,
    BuildOptionsResponse,
    BuildRequest,
    BuildSelectionConfirmationResponse,
    ai_pending_response,
    classify_game_direction,
    match_build_template,
    rank_build_templates,
    rules_fallback_response,
    template_response,
)
from app.catalog.repository import (
    get_components_by_ids,
    list_component_prices,
```

```

list_components,
list_gpu_whitelist_prices,
)
from app.compat.engine import BuildSelection, evaluate_compatibility
from app.core.rate_limit import high_cost_rate_limit
from app.db import get_session
router = APIRouter(
    prefix="/v1/build",
    tags=["build"],
    dependencies=[Depends(high_cost_rate_limit)],
)
logger = logging.getLogger(__name__)
BUILD_OPTION_MODES = ("used", "new", "mixed")
BUILD_DIRECTION_TOKENS = {"fps": "FPS", "aaa": "3A", "balanced": "均衡"}
BUILD_PURCHASE_TOKENS = {"used": "二手", "new": "全新", "mixed": "混合采购"}
BUILD_GPU_VENDOR_TOKENS = {"nvidia": "NVIDIA", "amd": "AMD", "intel": "Intel"}
@router.post("/options", response_model=BuildOptionsResponse)
def get_build_options(
    http_request: Request,
    request: BuildRequest,
    session: Session = Depends(get_session),
) -> BuildOptionsResponse:
    direction = request.direction or classify_game_direction(request.game_categories)
    request = request.model_copy(update={"direction": direction})
    templates = list_build_templates(session)
    components = list_components(session) if request.requires_customization else []
    prices = list_component_prices(session) if request.requires_customization else []
    components_by_id = {component.id: component for component in components}
    price_by_component_id = {price.component_id: price for price in prices}
    request_hash = request_payload = cache_version = None
    if request.requires_customization:
        request_hash, request_payload = request_identity(request)
        cache_version = current_cache_version(session)
        if request.budget > 5_000 and not request.no_gpu_build:
            excluded_gpu_ids = {
                row.component_id
                for row in list_gpu_whitelist_prices(session)
                if not gpu_brand_allowed_for_budget(row.name, request.budget)
            }
            components = [
                component
                for component in components
                if component.id not in excluded_gpu_ids
            ]
            components_by_id = {component.id: component for component in components}
    if request.no_gpu_build:
        try:
            find_owned_gpu(request.owned_gpu_model or "", components)
        except CustomizationError as exc:
            raise HTTPException(status_code=422, detail=str(exc)) from exc

```

```

options = []
unavailable_modes = []
for purchase_mode in BUILD_OPTION_MODES:
    mode_option_count = 0
    for gpu_vendor in _option_gpu_vendors(request, direction):
        cache_key = option_cache_key(purchase_mode, gpu_vendor)
        forced_request = request.model_copy(
            update={
                "use_case": "办公" if request.use_case == "办公" else "游戏",
                "preferences": [
                    BUILD_DIRECTION_TOKENS[direction],
                    BUILD_PURCHASE_TOKENS[purchase_mode],
                ],
                "purchase_preference": None,
                "gpu_preference": {
                    BUILD_GPU_VENDOR_TOKENS[gpu_vendor] if gpu_vendor else None
                },
            }
        )
        if request.requires_customization:
            cached = selected_option(
                session,
                forced_request,
                request_hash,
                cache_key,
                cache_version,
                purchase_mode=purchase_mode,
                gpu_vendor=gpu_vendor,
            )
            if cached is not None:
                options.append(cached)
                mode_option_count += 1
                if request.ray_tracing is not False:
                    break
            continue
        candidates = customization_candidates(
            forced_request,
            templates,
            components_by_id,
            price_by_component_id,
            purchase_mode=purchase_mode,
            gpu_vendor=gpu_vendor,
        )
        option = _customized_option(
            http_request,
            forced_request,
            candidates,
            components_by_id,
            price_by_component_id,
        )

```

```

        if option is not None:
            option = store_pending_option(
                session,
                request_hash,
                cache_key,
                request_payload,
                cache_version,
                option,
            )
            options.append(option)
            mode_option_count += 1
        else:
            option = _base_option(forced_request, templates, session)
            if option is not None:
                options.append(option)
                mode_option_count += 1
            if mode_option_count and request.ray_tracing is not False:
                break
        if mode_option_count == 0:
            unavailable_modes.append(purchase_mode)
    if not options:
        raise HTTPException(
            status_code=503,
            detail="当前预算和游戏方向没有可用的结构化配置方案。",
        )
    return BuildOptionsResponse(
        direction=direction,
        options=options,
        unavailable_modes=unavailable_modes,
    )

@router.post(
    "/options/{selection_id}/select",
    response_model=BuildSelectionConfirmationResponse,
)
def select_build_option(
    selection_id: UUID,
    session: Session = Depends(get_session),
) -> BuildSelectionConfirmationResponse:
    row = mark_option_selected(
        session,
        str(selection_id),
        current_cache_version(session),
    )
    if row is None:
        raise HTTPException(status_code=404, detail="配置选择记录不存在或已失效。")
    return BuildSelectionConfirmationResponse(
        selection_id=row.id,
        selected_count=row.selected_count,
    )

def _base_option(

```

```

request: BuildRequest,
templates: list,
session: Session,
) -> Optional[BuildOptionResponse]:
    for template in rank_build_templates(request, templates):
        if not template.details:
            continue
        components = get_components_by_ids(session, template.components.values())
        compatibility = evaluate_compatibility(
            BuildSelection(components=dict(template.components)),
            {component.id: component for component in components},
        )
        if not compatibility.compatible:
            continue
        try:
            return BuildOptionResponse.model_validate(
                template_response(template, compatibility).model_dump()
            )
        except ValidationError as exc:
            logger.warning("Skipping invalid build option template %s: %s", template.id, exc)
    return None

def _customized_option(
    http_request: Request,
    request: BuildRequest,
    candidates: list,
    components_by_id: dict,
    price_by_component_id: dict,
) -> Optional[BuildOptionResponse]:
    fallback = deterministic_customization(
        request,
        candidates,
        components_by_id,
        price_by_component_id,
    )
    if not candidates or not http_request.app.state.settings.ai_provider_api_key:
        return fallback
    candidates_by_id = {candidate.id: candidate for candidate in candidates}
    retry_feedback = None
    for _ in range(2):
        try:
            selection = select_build_with_deepseek(
                request,
                candidates,
                components_by_id,
                price_by_component_id,
                http_request.app.state.settings,
                retry_feedback=retry_feedback,
            )
            http_request.app.state.high_cost_usage_metrics.record_actual_ai_cost(
                http_request.url.path,

```



```

        selection.actual_cost_cents,
    )
    selected_template = candidates_by_id[selection.base_template_id]
    try:
        return customize_template(
            request,
            selected_template,
            selection.patches,
            components_by_id,
            price_by_component_id,
            source="ai_provider",
            reasons=selection.reasons,
        )
    except CustomizationError:
        return customize_template(
            request,
            selected_template,
            {},
            components_by_id,
            price_by_component_id,
            source="ai_provider",
            reasons=selection.reasons,
        )
    except (AIProviderError, CustomizationError, KeyError) as exc:
        retry_feedback = str(exc)[:240]
    http_request.app.state.high_cost_usage_metrics.record_external_ai_failure(
        http_request.url.path,
    )
    return fallback

def _option_gpu_vendors(
    request: BuildRequest,
    direction: str,
) -> tuple[Optional[str], ...]:
    if request.no_gpu_build:
        return (None,)
    if request.use_case == "办公":
        return (None,)
    if request.use_case == "游戏兼办公":
        return ("nvidia",)
    if request.budget < 5_000 or direction == "fps":
        return (None,)
    if request.ray_tracing is False:
        return ("nvidia", "amd")
    if request.ray_tracing is True:
        return ("nvidia",)
    return ("nvidia", "amd")

@router.post("/generate", response_model=BuildGenerationResponse)
def generate_build(
    http_request: Request,
    request: BuildRequest,

```

```

session: Session = Depends(get_session),
) -> BuildGenerationResponse:
    template = match_build_template(request, list_build_templates(session))
    if template is None:
        fallback = rules_fallback_response(
            request,
            list_components(session),
            list_component_prices(session),
        )
    if fallback is not None:
        return fallback
    return ai_pending_response(
        ai_provider_configured=bool(http_request.app.state.settings.ai_provider_api_key)
    )
    components = get_components_by_ids(session, template.components.values())
    compatibility = evaluate_compatibility(
        BuildSelection(components=dict(template.components)),
        {component.id: component for component in components},
    )
    return template_response(template, compatibility)
// File: backend/app/api/review.py
import json
from typing import Generator, Tuple
from fastapi import APIRouter, Depends, File, HTTPException, Request, Response, UploadFile
from fastapi.responses import StreamingResponse
from sqlalchemy.orm import Session
from app.core.rate_limit import high_cost_rate_limit
from app.core.response_cache import response_cache_key
from app.db import get_session
from app.review.ocr import OCRTextNotFoundError, OCRUnavailableError, extract_text_from_image_bytes
from app.review.service import (
    ConfigReviewRequest,
    ConfigReviewResponse,
    analyze_configuration_text,
)
MAX_REVIEW_IMAGE_BYTES = 8 * 1024 * 1024
SUPPORTED_REVIEW_IMAGE_TYPES = {"image/jpeg", "image/png", "image/webp"}
router = APIRouter(
    prefix="/v1/review",
    tags=["review"],
    dependencies=[Depends(high_cost_rate_limit)],
)
@router.post("/analyze", response_model=ConfigReviewResponse)
def analyze_review(
    payload: ConfigReviewRequest,
    http_request: Request,
    response: Response,
    session: Session = Depends(get_session),
) -> ConfigReviewResponse:
    result, cache_status = _cached_or_analyze(http_request, session, payload)

```

```

response.headers["X-Cache"] = cache_status

return result

@router.post("/analyze/image", response_model=ConfigReviewResponse)
async def analyze_review_image(
    http_request: Request,
    response: Response,
    image: UploadFile = File(...),
    session: Session = Depends(get_session),
) -> ConfigReviewResponse:
    if image.content_type not in SUPPORTED_REVIEW_IMAGE_TYPES:
        raise HTTPException(status_code=415, detail="仅支持 JPG、PNG 或 WebP 配置单图片")
    image_bytes = await image.read(MAX_REVIEW_IMAGE_BYTES + 1)
    if len(image_bytes) > MAX_REVIEW_IMAGE_BYTES:
        raise HTTPException(status_code=413, detail="图片不能超过 8MB")
    try:
        text = extract_text_from_image_bytes(image_bytes)
    except OCRUnavailableError as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc
    except OCRTextNotFoundError as exc:
        raise HTTPException(status_code=422, detail=str(exc)) from exc
    result, cache_status = _cached_or_analyze(http_request, session, ConfigReviewRequest(text=text))
    response.headers["X-Cache"] = cache_status
    return result

@router.post("/analyze/stream")
def analyze_review_stream(
    payload: ConfigReviewRequest,
    http_request: Request,
    session: Session = Depends(get_session),
) -> StreamingResponse:
    def event_stream() -> Generator[str, None, None]:
        yield _sse("progress", {"stage": "received"})
        yield _sse("progress", {"stage": "analyzing"})
        result, cache_status = _cached_or_analyze(http_request, session, payload)
        yield _sse("cache", {"status": cache_status})
        yield _sse("result", result.model_dump(mode="json"))
    return StreamingResponse(event_stream(), media_type="text/event-stream")

def _cached_or_analyze(
    http_request: Request,
    session: Session,
    payload: ConfigReviewRequest,
) -> Tuple[ConfigReviewResponse, str]:
    cache_key = response_cache_key("review.analyze", payload)
    if http_request.app.state.settings.response_cache_enabled:
        cached = http_request.app.state.response_cache.get(cache_key)
        if cached is not None:
            http_request.app.state.high_cost_usage_metrics.record_cache_status(
                http_request.url.path,
                "HIT",
            )
    return ConfigReviewResponse.model_validate(cached), "HIT"

```

UzBox ai 装机助手 V1.0

```

result = analyze_configuration_text(session, payload.text)
if http_request.app.state.settings.response_cache_enabled:
    http_request.app.state.response_cache.set(cache_key, result.model_dump(mode="json"))
http_request.app.state.high_cost_usage_metrics.record_cache_status(
    http_request.url.path,
    "MISS",
)
http_request.app.state.high_cost_usage_metrics.record_estimated_cost(
    http_request.url.path,
    http_request.app.state.settings.high_cost_estimated_cost_cents,
)
return result, "MISS"
def _sse(event: str, data: dict) -> str:
    encoded = json.dumps(data, ensure_ascii=False, separators=(",", ":"))
    return f"event: {event}\ndata: {encoded}\n\n"
// File: backend/app/api/perf.py
import json
from typing import Generator, Tuple
from fastapi import APIRouter, Depends, Request, Response
from fastapi.responses import StreamingResponse
from sqlalchemy.orm import Session
from app.core.rate_limit import high_cost_rate_limit
from app.core.response_cache import response_cache_key
from app.db import get_session
from app.perf.service import PerfEstimateRequest, PerfEstimateResponse, estimate_performance
router = APIRouter(
    prefix="/v1/perf",
    tags=["performance"],
    dependencies=[Depends(high_cost_rate_limit)],
)
@router.post("/estimate", response_model=PerfEstimateResponse)
def estimate_perf(
    payload: PerfEstimateRequest,
    http_request: Request,
    response: Response,
    session: Session = Depends(get_session),
) -> PerfEstimateResponse:
    result, cache_status = _cached_or_estimate(http_request, session, payload)
    response.headers["X-Cache"] = cache_status
    return result
@router.post("/estimate/stream")
def estimate_perf_stream(
    payload: PerfEstimateRequest,
    http_request: Request,
    session: Session = Depends(get_session),
) -> StreamingResponse:
    def event_stream() -> Generator[str, None, None]:
        yield _sse("progress", {"stage": "received"})
        yield _sse("progress", {"stage": "estimating"})
        result, cache_status = _cached_or_estimate(http_request, session, payload)

```

UzBox ai 装机助手 V1.0

```

        yield_sse("cache", {"status": cache_status})
        yield_sse("result", result.model_dump(mode="json"))
    return StreamingResponse(event_stream(), media_type="text/event-stream")

def _cached_or_estimate(
    http_request: Request,
    session: Session,
    payload: PerfEstimateRequest,
) -> Tuple[PerfEstimateResponse, str]:
    cache_key = response_cache_key("perf.estimate.v2", payload)
    if http_request.app.state.settings.response_cache_enabled:
        cached = http_request.app.state.response_cache.get(cache_key)
        if cached is not None:
            http_request.app.state.high_cost_usage_metrics.record_cache_status(
                http_request.url.path,
                "HIT",
            )
            return PerfEstimateResponse.model_validate(cached), "HIT"
    result = estimate_performance(session, payload)
    if http_request.app.state.settings.response_cache_enabled:
        http_request.app.state.response_cache.set(cache_key, result.model_dump(mode="json"))
    http_request.app.state.high_cost_usage_metrics.record_cache_status(
        http_request.url.path,
        "MISS",
    )
    http_request.app.state.high_cost_usage_metrics.record_estimated_cost(
        http_request.url.path,
        http_request.app.state.settings.high_cost_estimated_cost_cents,
    )
    return result, "MISS"

def _sse(event: str, data: dict) -> str:
    encoded = json.dumps(data, ensure_ascii=False, separators=(",", ":"))
    return f"event: {event}\ndata: {encoded}\n\n"

// File: backend/app/api/upgrade.py
import json
from typing import Generator, Tuple
from fastapi import APIRouter, Depends, Request, Response
from fastapi.responses import StreamingResponse
from sqlalchemy.orm import Session
from app.core.rate_limit import high_cost_rate_limit
from app.core.response_cache import response_cache_key
from app.db import get_session
from app.upgrade.service import UpgradePlanRequest, UpgradePlanResponse, generate_upgrade_plan

router = APIRouter(
    prefix="/v1/upgrade",
    tags=["upgrade"],
    dependencies=[Depends(high_cost_rate_limit)],
)

UPGRADE_CACHE_NAMESPACE = "upgrade.plan.v2"

@router.post("/plan", response_model=UpgradePlanResponse)
def plan_upgrade(

```

UzBox ai 装机助手 V1.0

```

payload: UpgradePlanRequest,
http_request: Request,
response: Response,
session: Session = Depends(get_session),
) -> UpgradePlanResponse:
    result, cache_status = _cached_or_plan(http_request, session, payload)
    response.headers["X-Cache"] = cache_status
    return result
@router.post("/plan/stream")
def plan_upgrade_stream(
    payload: UpgradePlanRequest,
    http_request: Request,
    session: Session = Depends(get_session),
) -> StreamingResponse:
    def event_stream() -> Generator[str, None, None]:
        yield _sse("progress", {"stage": "received"})
        yield _sse("progress", {"stage": "planning"})
        result, cache_status = _cached_or_plan(http_request, session, payload)
        yield _sse("cache", {"status": cache_status})
        yield _sse("result", result.model_dump(mode="json"))
    return StreamingResponse(event_stream(), media_type="text/event-stream")
def _cached_or_plan(
    http_request: Request,
    session: Session,
    payload: UpgradePlanRequest,
) -> Tuple[UpgradePlanResponse, str]:
    cache_key = response_cache_key(UPGRADE_CACHE_NAMESPACE, payload)
    if http_request.app.state.settings.response_cache_enabled:
        cached = http_request.app.state.response_cache.get(cache_key)
        if cached is not None:
            http_request.app.state.high_cost_usage_metrics.record_cache_status(
                http_request.url.path,
                "HIT",
            )
            return UpgradePlanResponse.model_validate(cached), "HIT"
    result = generate_upgrade_plan(session, payload)
    if http_request.app.state.settings.response_cache_enabled:
        http_request.app.state.response_cache.set(cache_key, result.model_dump(mode="json"))
        http_request.app.state.high_cost_usage_metrics.record_cache_status(
            http_request.url.path,
            "MISS",
        )
        http_request.app.state.high_cost_usage_metrics.record_estimated_cost(
            http_request.url.path,
            http_request.app.state.settings.high_cost_estimated_cost_cents,
        )
    return result, "MISS"
def _sse(event: str, data: dict) -> str:
    encoded = json.dumps(data, ensure_ascii=False, separators=(",", ":"))
    return f"event: {event}\ndata: {encoded}\n\n"

```